

Dynamic Vine Copula Risk Monitoring

1. Project Objective

Build a Python project that investigates how the **multivariate dependence structure between financial assets changes over time**, using rolling vine copula models.

The project should be positioned as a quantitative finance / risk-modelling project rather than a trading strategy.

The central research question is:

How does the dependence structure of a multi-asset portfolio evolve over time, and can meaningful changes in dependence and tail dependence be detected using rolling vine copula models?

A secondary objective is to investigate whether changes in dependence structure are associated with changes in portfolio risk measures such as Value at Risk (VaR) and Expected Shortfall (ES).

The project should ultimately support both:

1. historical analysis of financial dependence structures; and
2. a "live" monitoring mode in which new observations trigger updates to the estimated dependence structure.

2. Important Modelling Principle

Do **not** interpret every change in a fitted copula as a genuine structural change. Rolling estimates naturally fluctuate because the model is estimated using finite samples. Therefore, the project must distinguish between:

- normal estimation noise;
- gradual changes in dependence;
- meaningful structural changes/regime shifts.

The project should therefore explicitly separate:

Level 1 — Dependence monitoring

Track quantities such as:

- Pearson correlation
- Kendall's tau
- Spearman correlation
- lower-tail dependence
- upper-tail dependence

over time.

Level 2 — Vine structure monitoring

Track:

- selected pair-copula families;
- copula parameters;
- conditional dependencies;
- AIC/BIC or equivalent model-selection criteria;
- changes in the vine tree structure.

Level 3 — Structural change detection

Develop a method to identify whether the observed changes are large enough to constitute a potential dependence-regime change.

Do not assume that market crises necessarily produce a particular vine structure. The analysis should discover this from the data.

3. Financial Data

Initially use a small but meaningful set of liquid assets.

A possible initial universe is:

- AAPL
- MSFT
- NVDA
- JPM
- XOM
- JNJ
- SPY
- TLT

This universe should be configurable rather than hard-coded.

The implementation should allow the user to easily replace the assets with another list of tickers.

The project should initially use **daily data**.

Do not begin with high-frequency/intraday data. Daily data is much more appropriate for developing and validating the methodology before introducing the additional complications of microstructure noise, asynchronous observations, market hours, etc.

4. Price Data → Returns

Do not fit the copula directly to stock prices.

Use log returns:

$$\begin{aligned} &[\\ r_{\{i,t\}} = & \\ &\log\left(\frac{P_{\{i,t\}}}{P_{\{i,t-1\}}}\right) \\ &] \end{aligned}$$

where:

- $(P_{\{i,t\}})$ is the adjusted price of asset (i) at time (t);
- $(r_{\{i,t\}})$ is its log return.

Use appropriately adjusted historical prices where available so that corporate actions such as splits and dividends do not introduce artificial movements.

The data-processing pipeline should therefore be:

Price data



Cleaning



Adjusted prices



Log returns



Missing-data handling



Model-ready return matrix

The return matrix should have the form:

```
[  
R \in \mathbb{R}^{T \times d}  
]
```

where:

- (T) = number of observations;
- (d) = number of assets.

5. Marginal Distributions

Before fitting a copula, the marginal distributions need to be transformed to approximately uniform variables.

The initial implementation should support a simple empirical/rank-based probability integral transform.

For each asset:

```
[  
u_{i,t} \approx \frac{\operatorname{rank}(r_{i,t})}{T+1}  
]
```

so that:

```
[  
u_{i,t} \in (0,1).  
]
```

The project should be designed so that parametric marginal distributions can be added later.

Possible future marginal models include:

- Gaussian
- Student-t
- skewed distributions
- Generalized Hyperbolic
- GARCH-based conditional distributions

However, **do not implement all of these initially.**

Start with a robust empirical transformation.

The architecture should make it possible to replace the marginal model later without rewriting the copula engine.

6. Static Vine Copula

Before implementing rolling estimation, create a static vine copula model using a historical sample.

The purpose is to verify that:

1. the data pipeline works;
2. the marginal transformation works;
3. the vine copula can be fitted;
4. dependence measures can be extracted;
5. the resulting model can be visualized/interpreted.

The static model should be implemented as a separate, testable component.

For example:

```
vine = VineCopula(...)  
vine.fit(uniform_data)
```

```
results = vine.summary()
```

The exact Python package/library should be selected based on current package stability and documentation.

Do not invent a custom vine-copula implementation unless there is a compelling reason.

Prefer a mature, well-documented library.

7. Rolling Vine Copula

The main component of the project is a rolling estimation framework.

Suppose there are (T) observations and a rolling window of (W) observations.

At time (t), estimate:

```
\operatorname{Fit}(R_{t-W+1:t})
```

]

where (C_t) is the estimated multivariate copula for the current window.

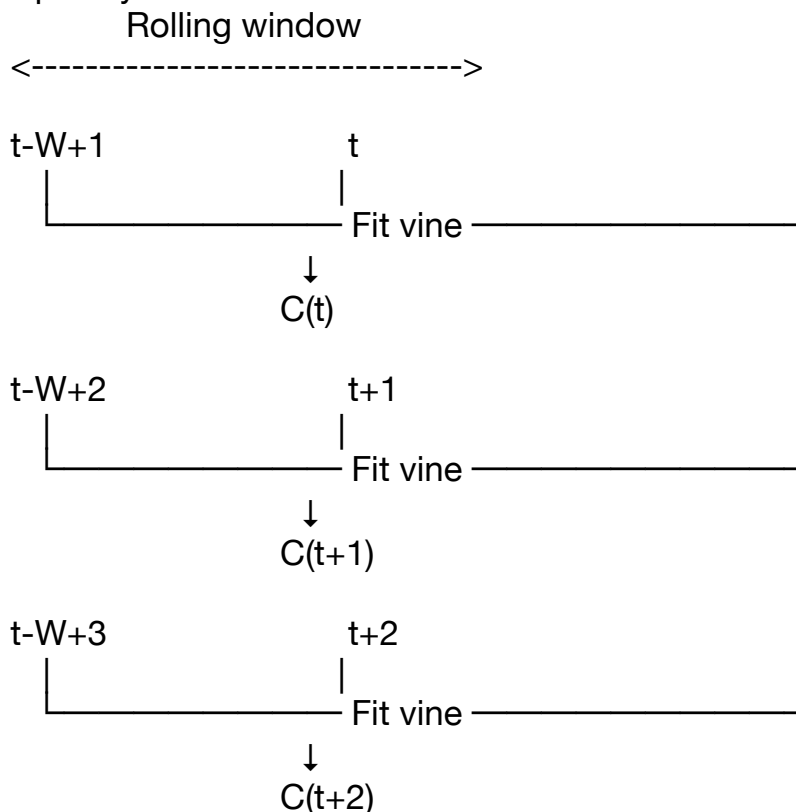
Then move the window forward:

```
\operatorname{Fit}(R_{t-W+2:t+1})
```

]

and continue through the dataset.

Conceptually:



The rolling window size must be configurable.

Initial candidate:

window = 250 trading days

but the implementation should allow:

window = 125

window = 250

window = 500

etc.

This will later allow sensitivity analysis.

8. Do Not Necessarily Refit Every Day

The framework should support daily rolling estimation, but the design should not assume that every observation must trigger a full expensive refit.

Potential approaches:

Option A — full daily refit

Simple and appropriate for the first implementation.

Option B — periodic refit

For example:

fit → wait 5 observations → refit → wait 5 observations → ...

Option C — online/update approximation

Potential future extension.

Start with **Option A** for correctness and simplicity.

The architecture should nevertheless separate:

data update

from:

model refit

so that alternative update frequencies can be introduced later.

9. What Should Be Stored From Each Vine Fit?

Do not simply store the fitted model object.

Create a structured results object for each timestamp.

At minimum, store:

Metadata

- timestamp
- window start
- window end
- number of observations
- number of assets

Pairwise dependence

For every asset pair:

- Kendall's tau
- Spearman correlation
- Pearson correlation

Copula information

For every pair-copula where available:

- copula family
- fitted parameters
- AIC
- BIC/log-likelihood where available

Tail dependence

Where supported:

- lower-tail dependence
- upper-tail dependence

Vine structure

Store enough information to reconstruct or visualize:

- tree structure
- edges
- conditional variables
- pair-copula families

Model diagnostics

Store:

- log-likelihood
- AIC
- BIC
- fitting status
- convergence/error information

The output should be serializable, for example to:

.parquet

or:

.json

where appropriate.

10. Measuring Dependence Changes

The project should not rely on one single metric.

Implement several complementary measures.

10.1 Average absolute Kendall dependence

For (d) assets:

```
[
D_t =
\frac{2}{d(d-1)}
\sum_{i<j} |\tau_{ij,t}|
]
```

This provides a simple overall measure of market interconnectedness.

Track (D_t) over time.

10.2 Pairwise dependence changes

For every pair:

```
\tau_{ij,t} - \tau_{ij,t-1}
]
```

This allows the system to identify which relationships changed.

10.3 Tail dependence

Track:

```
[
\lambda_L
]
```

and

[
 λ_U
]

where available.

This is particularly important because two assets can have similar ordinary correlations while having very different joint-tail behaviour.

The project should specifically investigate whether lower-tail dependence changes differently from ordinary correlation.

10.4 Vine structure changes

Track changes in:

- selected copula families;
- tree edges;
- conditional relationships.

A structural change should not simply mean:

"one copula parameter moved slightly."

Instead, investigate whether several aspects of the estimated model change simultaneously.

11. Structural Change Detection

This is one of the main research components.

The project should experiment with multiple approaches rather than immediately declaring one metric to be "the" structural-break measure.

Possible approaches include:

Approach 1 — parameter distance

Define a distance between consecutive fitted models.

For example:

[
 $D(C_t, C_{t-1})$
]

where the exact distance depends on the representation of the vine.

Approach 2 — dependence-matrix distance

Construct:

[
 $T_t =$
 $[\tau_{ij,t}]$
]

and compare:

[
 $|T_t - T_{t-1}|_F$
]

using the Frobenius norm.

This is simple and interpretable.

Approach 3 — rolling baseline

Compare current dependence with a longer-term baseline:

$$\frac{D_t - \mu_{D,t}}{\sigma_{D,t}}$$

where the mean and standard deviation are calculated over a historical baseline.
This gives a standardized measure of unusual dependence.

Approach 4 – statistical change-point detection

Potential future methods:

- CUSUM
- Page-Hinkley
- Bayesian change-point detection
- PELT
- other appropriate change-point methods

Do not implement every method immediately.

Start with a simple interpretable baseline and then add more sophisticated methods.

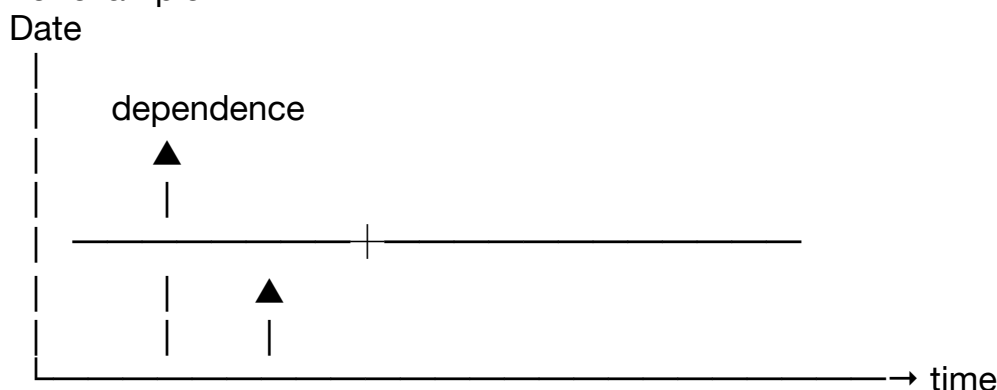
12. Market Regime Analysis

After building the rolling model, investigate whether detected dependence changes correspond to recognizable market periods.

Do not hard-code the conclusion that crises cause higher dependence.

Instead, identify historical periods and test whether the estimated dependence metrics changed.

For example:



Potential outputs:

- rolling average dependence;
- lower-tail dependence;
- number of structural changes;
- portfolio VaR;
- portfolio ES.

The analysis should explicitly distinguish **observed association** from causal claims.

13. Portfolio Risk

Once the dependence model works, use it to construct a portfolio risk analysis.

Given weights:

```
[  
w_1,\ldots,w_d  
]
```

and simulated returns:

```
[  
r_1,\ldots,r_d  
]
```

calculate portfolio return:

```
[  
R_p = \sum_{i=1}^d w_i r_i.  
]
```

Then estimate:

Value at Risk

```
[  
VaR_\alpha  
]
```

and:

Expected Shortfall

```
E[L\mid L>VaR_\alpha].  
]
```

The purpose is not to build a trading strategy.

The purpose is to investigate:

How does changing dependence affect estimated portfolio tail risk?

This creates a useful connection between the statistical model and practical risk management.

14. "Live" Monitoring Mode

After the historical implementation works, create a monitoring component.

The conceptual workflow is:

New market observation



Update price data



Calculate new return



Update rolling window



Refit/recalculate model



Calculate dependence metrics



Compare with previous model



Structural-change score



Monitoring output

For the first version, this does **not** need to connect to a real-time broker or trading system.

A simulated live mode using historical data is preferable.

For example:

for date in historical_dates:

```
    update_window(date)
```

```
    if model_should_update(date):
```

```
        model = fit_vine(window)
```

```
        metrics = calculate_metrics(model)
```

```
        alert = detect_change(metrics)
```

```
        dashboard.update(metrics, alert)
```

This allows the entire system to be tested reproducibly.

15. Dashboard

Create a simple dashboard only after the underlying analysis is working.

Potential technology:

- Streamlit

The dashboard should display:

Panel 1 — Asset prices/returns

Time series of selected assets.

Panel 2 — Rolling dependence

```
[  
D_t  
]
```

over time.

Panel 3 — Pairwise dependence

Heatmap of:

```
[  
\tau_{ij,t}  
]
```

for a selected date.

Panel 4 — Dependence evolution

Select two assets and display:

```
[  
\tau_{ij,t}  
]
```

through time.

Panel 5 — Tail dependence

Plot:

```
[  
  \lambda_L(t)  
]
```

and:

```
[  
  \lambda_U(t)  
]
```

where available.

Panel 6 — Vine structure

Visualize the selected vine tree structure.

Panel 7 — Structural-change score

Show:

```
[  
  S_t  
]
```

and highlight periods where the score exceeds a configurable threshold.

Panel 8 — Portfolio risk

Show rolling:

- VaR
- Expected Shortfall

16. Software Architecture

The project should be implemented as a proper Python package rather than one large notebook.

Suggested structure:

dynamic-vine-copula-risk/

```
|  
|—— README.md  
|—— pyproject.toml  
|—— LICENSE  
|—— .gitignore  
|  
|—— src/  
|   |—— vine_risk/  
|       |—— __init__.py  
|       |  
|       |—— data.py  
|       |—— returns.py  
|       |—— marginals.py  
|       |—— copula.py  
|       |—— rolling.py
```

```

|   |—— dependence.py
|   |—— change_detection.py
|   |—— portfolio.py
|   |—— diagnostics.py
|   |—— visualization.py
|
|—— notebooks/
|   |—— 01_data_exploration.ipynb
|   |—— 02_static_vine.ipynb
|   |—— 03_rolling_vine.ipynb
|   |—— 04_dependence_changes.ipynb
|   |—— 05_portfolio_risk.ipynb
|
|—— tests/
|   |—— test_returns.py
|   |—— test_marginals.py
|   |—— test_dependence.py
|   |—— test_portfolio.py
|
|—— dashboard/
|   |—— app.py
|
|—— data/
|   |—— README.md
|
|—— reports/
|   |—— figures/

```

17. Software Engineering Requirements

The project should demonstrate good programming practices.

Use:

- type hints;
- docstrings;
- modular functions/classes;
- unit tests;
- logging;
- configuration rather than hard-coded parameters;
- reproducible experiments;
- clear error handling.

Avoid putting the entire analysis in one notebook.

Notebooks should primarily demonstrate and visualize functionality that exists in the package.

18. Configuration

Important parameters should be configurable.

For example:

assets:

- AAPL
- MSFT
- NVDA
- JPM
- XOM
- JNJ
- SPY
- TLT

data:

frequency: daily

rolling:

window: 250

refit_frequency: 1

risk:

confidence_level: 0.99

simulations: 10000

The exact configuration format can be chosen by the developer.

19. Testing Strategy

The project should contain tests for mathematical/statistical functionality.

Examples:

Return calculation

Verify:

```
[  
r_t=\log(P_t/P_{t-1})  
]
```

against known values.

Rank transformation

Verify that:

```
[  
u_i\in(0,1)  
]
```

and that ranks are correctly assigned.

Dependence metrics

Compare calculated Kendall's tau against a trusted implementation.

Portfolio loss

Verify:

```
[  
L = -w^T r  
]
```

against manually calculated examples.

Change detection

Create synthetic data with a known dependence change and verify that the detector responds more strongly around the known change point.

This last test is particularly important.

20. Synthetic Data Validation

Before trusting the method on real financial data, create synthetic experiments.

For example:

Experiment A — constant dependence

Generate observations from a constant copula:

```
[  
C_t=C  
]
```

throughout the entire sample.

The change detector should generally **not** identify persistent structural breaks.

Experiment B — known dependence change

Generate:

```
[  
C_t =  
\begin{cases}  
C_1 & t < T_0 \\  
C_2 & t \geq T_0  
\end{cases}  
]
```

where (C_1) and (C_2) have different dependence parameters.

The detector should identify a change around (T_0) .

Experiment C — tail dependence change

Keep ordinary dependence relatively similar while changing tail dependence.

This tests whether the system can detect changes that ordinary correlation misses.

Synthetic validation should come before interpreting results from real stocks.

21. Important Statistical Caveats

The README should explicitly discuss limitations.

Potential issues include:

Rolling-window dependence

A rolling window means the fitted model represents a local historical distribution, not necessarily a stationary global distribution.

Model-selection instability

Different vine structures can have similar likelihoods.

Multiple testing

If many pairs and many dates are examined, some apparently unusual changes will occur by chance.

Financial non-stationarity

Financial return distributions change over time.

Marginal misspecification

Incorrect marginal models can affect the copula estimation.

Data frequency

Daily data does not capture intraday dependence dynamics.

Look-ahead bias

The live monitoring system must only use observations available up to the relevant timestamp.

Structural-change interpretation

A detected statistical change should not automatically be interpreted as a fundamental economic regime change.

22. Reproducibility

The project should make it possible to reproduce the analysis from scratch.

The README should contain instructions similar to:

```
git clone <repository>
```

```
cd dynamic-vine-copula-risk
```

```
python -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install -e .
```

```
pytest
```

```
python ...
```

The exact commands should depend on the final project structure.

Use a fixed random seed wherever simulation is involved.

23. Development Order

Do not implement the entire project at once.

Build it in the following order.

Phase 1 — Data

Implement:

- downloading/loading price data;
- cleaning;
- adjusted prices;
- log returns;
- basic visualizations.

Deliverable:

clean return matrix

Phase 2 — Marginals

Implement:

- empirical CDF/rank transformation;
- validation plots;
- interface allowing future parametric marginals.

Deliverable:

uniform_data

Phase 3 — Static Vine

Implement:

- vine fitting;
- model diagnostics;
- dependence extraction;
- structure visualization.

Deliverable:

fitted_static_vine

Phase 4 — Rolling Model

Implement:

RollingVineModel(...)

with configurable:

- window size;
- refit frequency;
- assets.

Deliverable:

timestamp → fitted model metrics

Phase 5 — Dependence Monitoring

Implement:

- rolling Kendall's tau;
- pairwise changes;
- average dependence;
- tail dependence;
- vine structure changes.

Deliverable:

dependence_metrics.parquet

Phase 6 — Structural Change Detection

Start with a simple interpretable metric.

For example:

```
[  
S_t=|T_t-T_{t-1}|_F  
]
```

Then investigate more sophisticated change-point methods.

Deliverable:

timestamp → structural_change_score

Phase 7 — Portfolio Risk

Implement:

- copula-based simulation;
- portfolio returns;
- VaR;
- Expected Shortfall.

Investigate how portfolio risk changes alongside dependence.

Phase 8 — Synthetic Validation

Test the methodology on simulated data with known dependence structures and known change points.

This should be treated as an essential validation step rather than an optional demonstration.

Phase 9 — Dashboard

Only after the modelling pipeline is reliable, create the Streamlit dashboard.

Phase 10 — Live Simulation

Replay historical observations sequentially and simulate the monitoring system receiving new observations over time.

This provides a realistic test of the "live" architecture without requiring a real-time market-data provider.

24. What the Final README Should Communicate

The README should not simply say:

"This project uses vine copulas to analyze stocks."

Instead, present it as a quantitative modelling problem.

Suggested opening:

Dynamic Vine Copula Risk Monitoring

A quantitative risk framework for estimating and monitoring time-varying dependence structures in financial markets using rolling vine copula models.

The project investigates how pairwise and higher-order dependence, particularly tail dependence, evolves over time and whether statistically meaningful changes in the dependence structure can be detected. The resulting dependence models are subsequently used to investigate implications for portfolio tail risk.

Then show:

1. methodology;
2. architecture;
3. example dependence evolution;
4. structural-change detection;
5. portfolio-risk implications;
6. limitations.

25. What NOT to Do Initially

Avoid:

- deep learning;

- reinforcement learning;
 - automated trading;
 - intraday data;
 - dozens of assets;
 - complicated marginal distributions;
 - real-time broker integration;
 - dozens of change-point algorithms;
 - an elaborate dashboard before the statistical model works.
- The goal is to create a **small, mathematically rigorous and well-engineered research project** first.
Complexity can be added after the core methodology is validated.

26. Desired End Result

The final project should demonstrate competence in:

Mathematics

- probability
- multivariate distributions
- copulas
- dependence modelling
- statistical estimation
- Monte Carlo simulation
- optimization
- risk measures

Statistics

- marginal modelling
- dependence estimation
- model selection
- uncertainty
- change-point detection
- model validation

Programming

- Python package development
- object-oriented/modular design
- testing
- data pipelines
- numerical computing
- visualization
- reproducibility

Quantitative finance

- portfolio modelling
- tail risk
- VaR
- Expected Shortfall
- dependence risk
- stress/regime analysis

27. Instructions to the Implementing LLM

Build the project **incrementally**.

Do not immediately generate hundreds of lines of code.

First:

1. inspect the current environment;
2. determine suitable Python packages for vine copulas;
3. propose the package choices and explain any trade-offs;
4. create the repository structure;
5. implement the data and return pipeline;
6. write tests;
7. verify that the first stage works;
8. then proceed to the next stage.

At every stage, prioritize:

- statistical correctness;
- reproducibility;
- modularity;
- testability;
- clear documentation.

When there is a choice between a sophisticated method and a simpler method that is easier to validate, implement the simpler method first and leave a clear extension point for the sophisticated version.

The ultimate objective is **not to produce the most complicated model**.

The objective is to produce a project where every modelling decision can be explained mathematically and empirically.